IMAGE SUPER-RESOLUTION USING SRCNN

A Machine Learning Mini-Project Report

Project: Single-Image Super-Resolution with a NumPy-based SRCNN implementation

Implementation: Python and NumPy

Abstract

Image super-resolution aims to create a higher-resolution image from a lower-resolution input. This project implements a compact Super-Resolution Convolutional Neural Network (SRCNN) using Python and pure NumPy. The system first enlarges a low-resolution image with bicubic interpolation, then applies three convolutional layers to learn a refined reconstruction. Training uses mean squared error (MSE), backpropagation, and the Adam optimizer. Model quality is evaluated with peak signal-to-noise ratio (PSNR). The supplied experiment results report 37.848 dB test PSNR for the residual SRCNN variant, compared with 36.857 dB for bicubic interpolation and 36.567 dB for the basic SRCNN configuration. These results are promising within the project setup, but should be interpreted cautiously because the included dataset contains 100 small image crops and the reported study uses a single random seed.

Keywords: Image super-resolution, SRCNN, convolutional neural network, NumPy, bicubic interpolation, MSE, PSNR, residual learning.

1. Introduction

Digital images represent scenes as grids of pixels. When an image has too few pixels, fine details such as edges, textures, and small structures may be lost. Ordinary resizing methods estimate new pixels from nearby existing pixels. Super-resolution goes further by learning patterns from training examples and using them to estimate a sharper reconstruction. This project is a from-scratch re-implementation inspired by SRCNN. The network, backpropagation routines, and Adam optimizer are implemented in NumPy rather than a deep-learning framework.

1.1 Problem statement

Given a low-resolution image, produce an image at twice its width and height that is closer to the corresponding high-resolution target than a simple interpolation baseline.

1.2 Project objectives

- Build a compact three-layer SRCNN-style model using NumPy.
- Create paired low-resolution inputs and high-resolution targets from the supplied image dataset.
- Implement forward propagation, MSE loss, backpropagation, and Adam parameter updates.
- Evaluate output quality using PSNR and compare learned models with interpolation baselines.
- Investigate residual learning, regularization, output activation, and batch normalization.

2. Background and Core Concepts

2.1 Image upscaling and super-resolution

Upscaling increases the dimensions of an image, but it cannot recover information that is no longer present in the low-resolution pixels with certainty. Bilinear interpolation uses weighted averages of nearby pixels; bicubic interpolation uses a wider neighborhood and cubic weighting. SRCNN learns a data-driven correction based on patterns found in training images. Its output is an estimate, not a guarantee of the exact original detail.

2.2 Convolutional neural networks

A convolutional layer applies small learnable filters across an image. Each filter responds to patterns such as edges, corners, or textures. Multiple feature maps allow the model to represent different visual patterns. ReLU, defined as max(0, x), introduces non-linearity so stacked layers can learn more complex mappings.

2.3 SRCNN layers

Stage	Configuration	Purpose
Input	224 × 224 × 3	Bicubic-upsampled RGB image
Convolution 1 + ReLU	9 × 9; 3 → 64 channels	Extract local patches and visual features
Convolution 2 + ReLU	1 × 1; 64 → 32 channels	Map features into a compact representation
Convolution 3	5 × 5; 32 → 3 channels	Reconstruct the RGB output
Output	224 × 224 × 3	Refined high-resolution estimate

The network is fully convolutional: the same learned filters can be applied to whole images of compatible dimensions, not only to training patches.

3. Dataset and Preprocessing

The bundled dataset contains 100 RGB high-resolution image crops of size 224 × 224. The README identifies them as crops from sample photographs distributed with scikit-image, rather than the full DIV2K benchmark.

Subset	Images	Purpose
Training	60	Update model parameters
Validation	20	Select best checkpoint and compare configurations
Testing	20	Estimate performance on held-out images

3.1 Creating paired examples

1. Start with a 224 × 224 high-resolution image as the target.
2. Downsample it to 112 × 112 to simulate a 2× lower-resolution input.
3. Upsample the small image back to 224 × 224 using bicubic interpolation.
4. Use the enlarged image as the SRCNN input and the original image as the training target.
5. During training, sample 48 × 48 patches and use image flips for data augmentation.

The split is made by source photograph to reduce overlap of visual content between training, validation, and test subsets. A separate preparation script is provided for replacing the bundled sample images with 100 centre-cropped DIV2K training images.

4. Methodology and Training

4.1 Forward propagation

The bicubic-upsampled image passes through the first convolution and ReLU, then the second convolution and ReLU, and finally the reconstruction convolution. The resulting three-channel image is the model prediction.

4.2 Loss function

The model is trained with mean squared error (MSE):

$$\text{MSE} = \text{mean}((\text{prediction} - \text{target})^2)$$

MSE penalizes pixel differences by squaring them before averaging. A lower value means that predicted pixel values are closer to target values.

4.3 Backpropagation and Adam

Backpropagation applies the chain rule to compute how the loss changes with each weight and bias. The gradients are then used by Adam, an adaptive optimization algorithm that maintains running estimates of gradients and squared gradients. The project includes a finite-difference gradient test to check whether the hand-written derivatives are numerically consistent.

4.4 Checkpoint selection

Validation PSNR is tracked during training, and the weights from the best validation epoch are retained. The training scripts expose options such as epoch count and learning rate.

5. Residual SRCNN Refinement

The project also evaluates a residual variant. Rather than learning the complete high-resolution image directly, the network predicts a correction that is added to the bicubic input:

$$\text{Final output} = \text{bicubic-upsampled input} + \text{learned residual}$$

The bicubic image already contains the broad scene structure. Residual learning lets the model focus on correcting errors and refining details. In the supplied experiment table, this variant has the highest test PSNR among the listed configurations.

6. Evaluation Metrics

6.1 Peak Signal-to-Noise Ratio (PSNR)

PSNR expresses reconstruction error on a logarithmic decibel scale. For images normalized to [0, 1], the project uses:

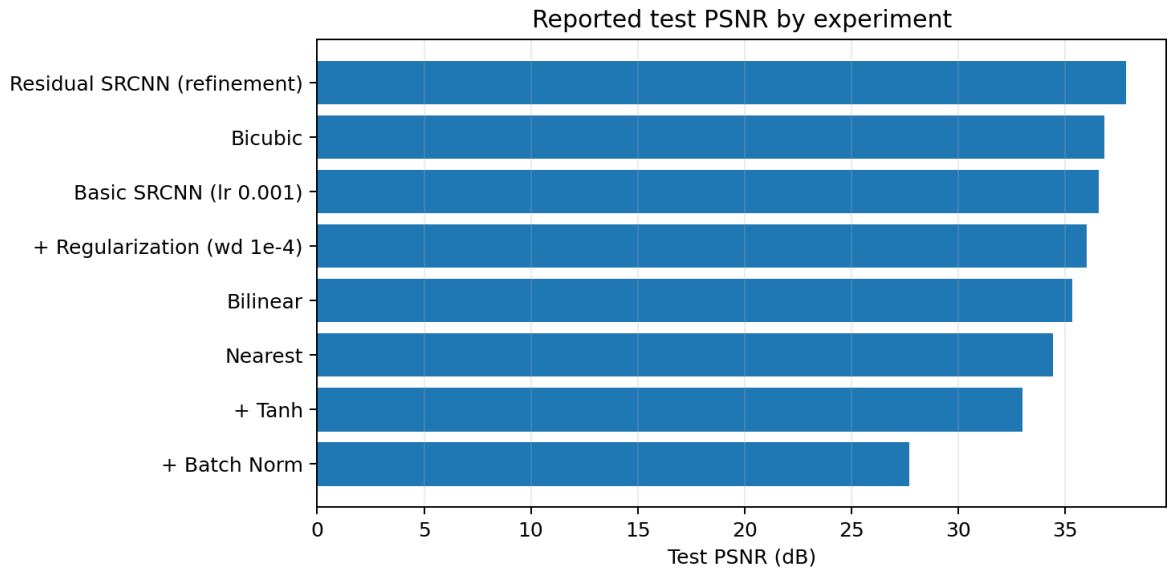
$$PSNR = 10 \times \log_{10}(1 / MSE)$$

Higher PSNR generally indicates lower pixel-level error. However, PSNR does not perfectly match human perception: a higher value does not automatically mean that every image looks sharper or more natural.

7. Experimental Results

The following values are taken from the project's supplied results_table.csv. Test PSNR is reported in decibels (dB).

Experiment	Max val PSNR	Test loss	Test PSNR
Residual SRCNN (refinement)	31.785	0.000432	37.848
Bicubic	29.866	0.000734	36.857
Basic SRCNN (lr 0.001)	30.943	0.000538	36.567
+ Regularization (wd 1e-4)	30.881	0.000585	36.019
Bilinear	28.901	0.000937	35.351
Nearest	28.833	0.000946	34.432
+ Tanh	28.717	0.000848	33.027
+ Batch Norm	27.197	0.001725	27.733



7.1 Key observations

- Residual SRCNN achieved the best reported test PSNR: 37.848 dB.
- Bicubic interpolation achieved 36.857 dB, while basic SRCNN achieved 36.567 dB on the test set.
- The residual variant improved test PSNR by about 0.991 dB over bicubic interpolation in this experiment.
- Regularization, Tanh output, and Batch Normalization did not improve the reported test result under the settings tested.
- The basic SRCNN result is slightly below bicubic on test, despite better validation performance.

7.2 Interpretation

Within this experiment, residual learning is the strongest configuration. However, the result should not be treated as proof that SRCNN will outperform bicubic interpolation on all images. The test set contains only 20 images, the study uses a single seed, and small PSNR differences are not statistically established. The values are also not directly comparable with results reported on the full DIV2K benchmark.

8. Software Structure and How to Run

The project is organized into model/data/training modules, scripts, tests, and precomputed results.

Path	Role
srcnn/core.py	SRCNN layers, forward/backward passes, BatchNorm, and Adam
srcnn/data.py	Image loading, low-resolution input generation, and PSNR
srcnn/trainer.py	Patch sampling, training loop, and evaluation
scripts/train.py	Train one model configuration
scripts/run_experiments.py	Run the experiment suite
scripts/demo.py	Run a model on an input image
tests/test_gradients.py	Numerical gradient check
results/	Saved weights, histories, tables, and figures

8.1 Setup

```
python -m venv venv

venv\Scripts\activate (Windows Command Prompt)

pip install -r requirements.txt

python tests/test_gradients.py
```

8.2 Train and run a demo

```
python scripts/train.py --epochs 30 --lr 0.001

python scripts/demo.py --input data/test/test_000_retina.png

python scripts/demo.py --input my_small_photo.png --no_degrade
```

Run commands from the project root after activating the environment. The demo can save side-by-side comparison images. With --no_degrade, the supplied image is treated as already low-resolution; PSNR cannot be calculated unless a matching high-resolution target is available.

9. Limitations

- Small dataset: 100 image crops total, including only 20 test images.
- Single-seed results: run-to-run variation has not been characterized.
- Dataset scale: the bundled images are not the full DIV2K benchmark.
- Metric limitations: PSNR measures pixel fidelity but does not fully describe perceptual quality.
- Pure NumPy is educational and portable, but generally slower than optimized GPU-based frameworks.
- Super-resolution estimates missing detail; it cannot guarantee exact recovery of information lost during downsampling.

10. Future Work

- Evaluate on a larger and standardized dataset such as DIV2K.
- Run multiple random seeds and report mean and standard deviation.

- Test additional scale factors, blur kernels, and realistic degradation conditions.
- Add visual crop comparisons and perceptual metrics alongside PSNR.
- Compare the NumPy implementation against an optimized framework implementation.
- Explore deeper architectures or perceptual-loss approaches while keeping a clear baseline.

11. Conclusion

This project demonstrates an end-to-end implementation of SRCNN-style image super-resolution in pure NumPy. It creates low-/high-resolution training pairs, learns a pixel reconstruction mapping using MSE and Adam, and evaluates output quality with PSNR. The experiment table shows that the residual refinement is the best-performing tested configuration, reaching 37.848 dB test PSNR. The results also highlight the importance of honest baselines, separate validation/test data, and careful interpretation of small-dataset metrics. A larger benchmark and repeated experiments are the most important next steps before drawing broader conclusions.

References

- Dong, C., Loy, C. C., He, K., & Tang, X. (2015). Image Super-Resolution Using Deep Convolutional Networks. IEEE Transactions on Pattern Analysis and Machine Intelligence, 38(2), 295–307.
- Garber, A., Grossman, T., & Johnson-Yu, S. Image Super-Resolution Via a Convolutional Neural Network. CS229 project report cited by the supplied README.
- Project source files and experiment outputs supplied in srcnn_mini_project(2).zip: README.md, results/results_table.csv, scripts/, srcnn/.